

OPERATING SYSTEM INSY 8223 - GROUP 3 (A+ REPORT)

Introduction

This report provides detailed, analytical, and applied answers to all questions from Lessons 5 and 6. Each answer includes reasoning, real-world interpretation, and structured diagrams where necessary.

1. Circular Wait Condition (Applied)

Circular wait is a deadlock condition where processes form a closed chain, each waiting for a resource held by the next process.

```
P1 holds R1, waits R2
P2 holds R2, waits R3
P3 holds R3, waits R1
```

```
Graph:
P1 → R2 → P2 → R3 → P3 → R1 → P1
```

This implies no process can proceed, creating a deadlock cycle.

2. Banker's Algorithm (Analytical)

The Banker's Algorithm ensures the system remains in a SAFE state by simulating allocation before granting resources.

If after allocation all processes can still finish, the system is safe. Otherwise, the request is denied.

This prevents deadlock by avoiding unsafe states.

3. User vs Kernel Threads (Analytical)

User-level threads are fast and managed in user space, but cannot achieve true parallelism.

Kernel-level threads are managed by OS and allow parallel execution across CPUs.

Therefore, kernel threads provide better system performance despite higher overhead.

4. pthread_join() (Applied)

```
pthread_create(&t1, ...);  
pthread_join(t1, NULL);
```

This ensures the main thread waits until t1 finishes. Without it, the program may terminate early.

5. Benefits of Multithreading (Conceptual)

Responsiveness: UI stays active (e.g., loading app data).

Resource Sharing: Threads use same memory space.

Parallelism: Multiple CPU cores execute tasks simultaneously.

6. Static vs Dynamic Loading (Analytical)

Static loading loads the entire program at once. Dynamic loading loads modules only when needed.

Dynamic loading is preferred when memory is limited or programs are large.

7. External Fragmentation (Analytical)

```
[P1][Free][P2][Free][Free][P3]
```

Free memory exists but is not contiguous.

Compaction rearranges memory to form one large block.

Cost: High CPU time and system overhead.

8. Single vs Multiple Partition Allocation (Analytical)

Single partition: one process occupies memory.

Multiple partition: memory split for multiple processes.

Thus, multiple partition is more flexible and efficient.

9. Page Fault (Conceptual)

A page fault occurs when required data is not in RAM.

Steps: detect → locate on disk → load → update table → resume execution.

10. Segmentation vs Paging (Analytical)

Segmentation is based on logical program structure and visible to the user.

Paging is fixed-size and hidden from the user.

Therefore, segmentation aligns with how programmers think, while paging optimizes memory management.